

Take-Home Architecture & Implementation Exercise

Senior Developer, Collaborate

Context

Caseware builds software that supports audit, assurance, risk, and compliance professionals in delivering high-quality work efficiently and consistently across their firm's practice. Collaborate is the product surface within our platform that lets engagement teams, client stakeholders, and third-party reviewers work together inside a shared engagement workspace — exchanging documents, comments, and financial data in real time, across firm and organizational boundaries.

Because workspaces routinely include people outside the firm (client employees, external auditors, regulators) as well as automated integrations (a client's own system calling in to pull data on a user's behalf), Collaborate needs an identity and authorization layer that is standards-compliant (OAuth2 / OIDC), fast, and safe to extend as new client integrations are added.

This exercise evaluates architecture, judgment, and system design at a Senior level, with a focus on C#/.NET implementation.

The Problem

Collaborate needs to expose OAuth2/OIDC-compliant endpoints supporting three related use cases:

1. **Login** – Interactive authentication for two populations: firm staff (who authenticate against Caseware's central identity provider) and invited external client users (some of whom belong to firms that want to federate their own SAML/OIDC identity provider rather than use Caseware credentials). This should use an Authorization Code + PKCE flow, with per-firm client configuration.
2. **Permission checking** – Every request into a workspace's resources (documents, comments, financial data extracts) must be authorized against fine-grained permissions: a workspace-level role (owner/contributor/viewer), resource-level overrides (e.g., a single document shared with one external user only), and firm-level policy. This needs to run fast enough for real-time, collaborative traffic (tens of thousands of authorization checks per second across firms) without a full database round-trip on every request. Permission revocation (e.g., removing an external user from a workspace) must take effect quickly — target within seconds — even though access tokens may still be valid and connections may be long-lived (e.g., an open collaborative editing session).
3. **On-behalf-of authorization** – Two delegation scenarios: (a) a client's own system calls into Collaborate's API on behalf of one of their employees (e.g., to pull specific engagement data into their internal system), and (b) internal Collaborate services call other internal Caseware APIs on behalf of the user who triggered an action (e.g., a notification service acting after a comment is posted), so the downstream call is scoped to what that specific user is allowed to do and remains attributable to them for audit purposes.

Consider how to avoid a “confused deputy” problem in the on-behalf-of flows, how to keep permission checks both fast and consistent with the source of truth, and how to handle revocation for long-lived sessions without forcing mass re-authentication.

Actually implementing the full identity provider (e.g., user credential storage, MFA) is outside the scope of this assignment — you can assume you're building the authorization layer around it.

Context (Other systems you can work with)

- Caseware's central identity provider issues base OIDC identity tokens for firm staff and can be assumed to expose standard OIDC discovery, token, and userinfo endpoints. You may treat it as an external dependency you call, not one you build.
- Some firms bring their own SAML/OIDC identity provider that needs to be federated in as an additional login option, scoped to their own users/workspaces.

- Workspace roles, resource-level permission overrides, and firm policy are stored in Collaborate's own database. You may assume you can add hooks/events to any permission or role change made in this database.
- Downstream resource APIs (Document Service, Financial Data API, Comments Service) each expect a validated access token containing specific scopes/claims and reject requests that don't have them — they do not talk to the permissions database directly.
- You may assume a reliable, low-latency cache/store (e.g., Redis) is available if your design calls for one.

Part 1: Architecture & Design (Primary Focus)

Please provide a short design document (1-3 pages max) that covers:

1. High-Level Architecture
2. Implementation Plan
3. Testing Strategy
4. Evaluation & Observability
5. Failure Modes & Tradeoffs

Part 2: Targeted Implementation

Optionally (but encouraged), implement one small, working slice of the authorization layer in C#/.NET (ASP.NET Core preferred). Choose ONE outcome to demonstrate:

A. A resource endpoint that only serves a request when the caller's token carries the correct scope/claims for that resource, and rejects it appropriately otherwise.

B. An endpoint that reports what the current user is authorized to access, Usable by another service that wouldn't have to compute authorization itself.

C. An endpoint that takes a caller's token and issues a new, narrower token scoped to a specific downstream user — solving the on-behalf-of problem described in Part 1.

We intentionally aren't telling you how to build these. Part of what's being evaluated is whether you reach for the right tool: ASP.NET Core's authentication/authorization framework already solves large parts of this, and using it well is often the correct senior move. You are not expected to hand-roll token parsing, signature verification, or key management unless you have a specific reason to — if you do write custom logic, briefly say why the built-in approach wasn't sufficient.

Guidelines:

- Pick one slice, not all three — depth over breadth.
- **Briefly justify the approach you chose (framework feature, custom code, or other) and its tradeoffs** — this matters as much as the code.
- Focus on interfaces, contracts, and correctness over completeness.
- Tests or validation logic are a plus.
- This is not a full application build, and you do not need a working identity provider behind it — stub or mock as needed.

AI Usage (Important)

You may use any AI tools you normally use (e.g., ChatGPT, Copilot, Claude).

During the follow-up review, we will ask you to explain:

- Where AI helped you

- Where you corrected or ignored AI output
- How you would guide other engineers using AI on this system
- Where AI should not be trusted in this domain

We are evaluating judgment and maturity, not just usage.

Assumptions & Constraints

- You may choose any cloud provider or stack for supporting infrastructure (note we are an AWS shop)
- Implementation, if attempted, should be in C#.NET (ASP.NET Core is a good default)
- You may state reasonable assumptions
- Optimize for clarity and tradeoffs over completeness

Deliverables

Please submit:

1. Design document (PDF or Markdown)
2. (Optional) Code or repository
3. (Optional) Session History for any AI tooling used
4. Any diagrams you created

What We're Evaluating

- Architecture and system design judgment for identity/authorization systems
- Working knowledge of OAuth2/OIDC concepts and protocols — and where you'd deviate from spec, and why
- Production and operational thinking (token lifecycle, revocation, caching, scale)
- **Judgment in choosing framework/library support vs. custom implementation** (Part 2)
- Clarity of communication
- C#.NET proficiency (if Part 2 attempted)
- Thoughtful use of AI in the Software Development Process

Time Expectation

We expect this exercise to take no more than 2–3 hours total.

Please do not over-optimize. This applies especially to Part 2: the right engineering choice (e.g., leaning on ASP.NET Core's built-in authentication/authorization framework rather than hand-rolling token parsing or cryptography) should keep you inside this budget. If you find yourself building cryptographic primitives from scratch, that's a signal to step back and reconsider the approach, not to push through.

Follow-Up

We will review your submission. If we like it, you will be invited to a live design review to discuss decisions, tradeoffs, and proposed implementation of the system.